

Agentic Coding Tools

Part 1: Introduction & Installation

J. Vaught

University of South Carolina

January 2026

A comprehensive guide to autonomous coding assistants
Covering 15+ tools and frameworks

Outline

- 1 What are Agentic Coding Tools?
- 2 The Three Categories Overview
- 3 Terminal Agents Installation
- 4 Agentic IDEs Installation
- 5 VS Code Plugins Installation
- 6 Summary & Next Steps

Understanding Agentic Coding Tools

Definition:

- Autonomous software agents using AI for code development
- Go beyond simple code completion—they **take actions**
- Read files, execute commands, run tests, install dependencies
- Multi-turn conversations with context awareness

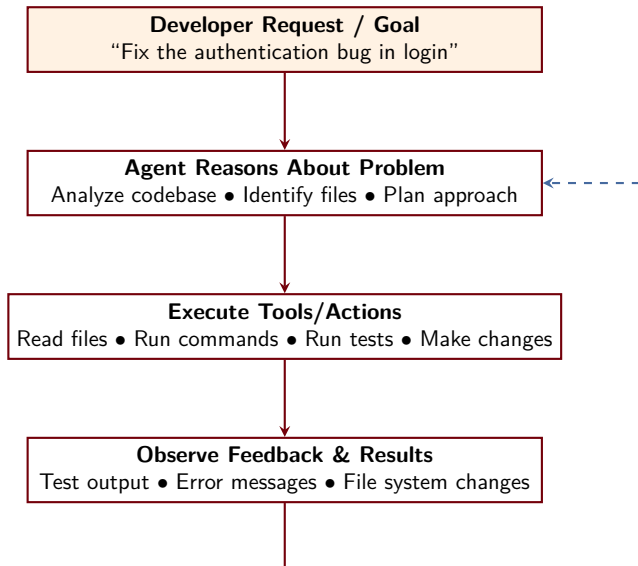
Key Characteristics

- ① **Autonomous Action**
- ② **Context Awareness**
- ③ **Tool Integration**
- ④ **Iterative Problem Solving**
- ⑤ **Multi-step Reasoning**

Paradigm Shift

Instead of copy-pasting suggestions, you **describe what you want** and the agent figures out how to accomplish it.

How Agentic Tools Work: The Agent Loop



Example: Fixing a Bug with an Agent

Scenario: “Fix the login endpoint—it’s returning 500 errors”

1. Exploration Phase (2–3 min)

- Search codebase for login files
- Examine error logs
- Identify dependencies
- Read authentication logic

2. Analysis Phase (1–2 min)

- Understand the problem
- Check recent commits
- Review test files

3. Implementation Phase (3–5 min)

- Propose fixes
- Write code changes
- Run tests automatically

4. Verification Phase (2–3 min)

- Execute full test suite
- Check for regressions
- Commit changes with git

Benefits of Agentic Coding Tools

Productivity Benefits

- 40–60% faster for certain tasks
- Eliminates context switching
- Automated testing/verification
- Faster debugging

Learning Benefits

- Step-by-step problem solving
- Best practices demonstrated
- Interactive feedback
- Explore alternatives

Quality Benefits

- Comprehensive code review
- Automated test generation
- Consistent code style
- Better documentation

Limitations

- Context window limits
- Hallucination risk
- API costs add up
- Learning curve varies

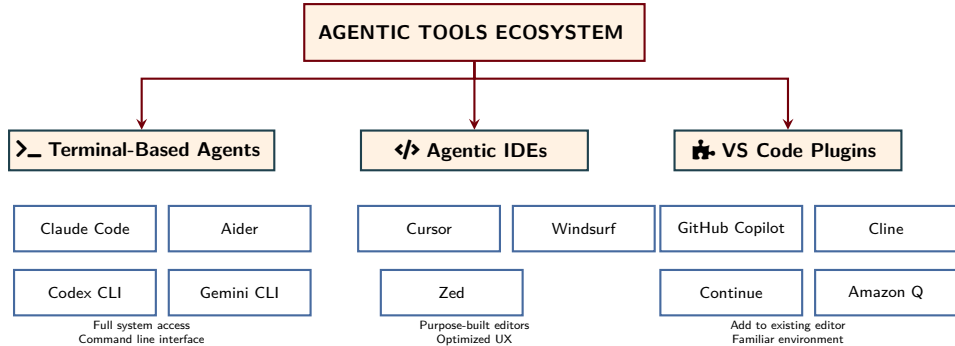
Traditional vs. Agentic Tools

Aspect	Traditional (Copilot)	Agentic (Claude Code, Aider)
Input	Type code, get suggestions	Describe task in natural language
Output	Code snippets	Complete solutions with verification
Scope	Single file, current context	Entire codebase
Actions	Suggest code	Execute commands, modify files, run tests
Verification	Manual testing	Automatic testing and validation
Error Recovery	You fix errors	Agent attempts fixes automatically
Use Case	Writing new code	Solving problems, refactoring, debugging

Complementary Tools

Agentic tools are **not replacements**—they're complementary. Use Copilot for autocomplete

The Agentic Tools Ecosystem



Category 1: Terminal-Based Agents

What Are They?

- Run in your terminal/command line
- Full access to system and codebase
- Natural language communication
- Execute any command you could
- Multi-turn conversations

Best For

- Complex multi-step tasks
- Automation and scripting
- Git integration
- Headless environments
- Development workflows

Why Most Powerful?

- 1 **Unrestricted Access**
- 2 **Scripting Integration**
- 3 **Version Control**
- 4 **Automation**

Trade-offs

- No visual IDE features
- Require terminal comfort
- Context shown in text only
- May need explicit instructions

Claude Code: Anthropic's Terminal Agent

Overview

- Developed by Anthropic
- Uses Claude language models
- Most sophisticated agentic capabilities
- MCP (Model Context Protocol) support

Best Use Cases

- Large refactoring projects
- Complex debugging
- Architectural discussions
- Building from specifications
- Learning and exploration

Key Strengths

- Best long-context understanding
- Excellent code analysis
- Strong reasoning for complex problems
- Multi-hour conversations
- Web search and tool use

Resource Requirements

- Terminal + API access
- Requires API key (paid)
- No heavyweight IDE needed

Aider: Open Source Powerhouse

Overview

- Open-source (GPLv3)
- Mature, well-maintained
- Multiple LLM providers
- Designed for pair programming

Supported Providers

- OpenAI (GPT-4, GPT-3.5)
- Anthropic Claude models
- **Local models via Ollama**
- Google Gemini
- Mistral, Cohere, others

Key Strengths

- Works **offline** with local models
- Multi-file editing with clear diffs
- Excellent git integration
- Active community
- Cost-effective options

Unique Features

- “Whole file” mode for rewrites
- Diff-based editing
- Built-in code review
- Completely offline operation

Codex CLI & Gemini CLI

Codex CLI (OpenAI)

Overview:

- OpenAI's official terminal tool
- Access to GPT-4 models
- Optimized for speed

Best For:

- Quick code generation
- Command-line tool building
- Script automation
- Fast feedback loops

Limitations:

- Less agentic than alternatives

Gemini CLI (Google)

Overview:

- Google's terminal interface
- Gemini 1.5 Pro/Flash models
- **1 million token context**

Best For:

- Large codebase analysis
- Multi-hour debugging
- Cost-conscious orgs
- Document-heavy workflows

Unique:

- 1M token context window

Other Terminal Agents (Brief Overview)

GitHub Copilot CLI

- GitHub ecosystem integration
- Good for GitHub workflows
- Fewer agentic features

Goose

- Lightweight open-source agent
- Python-based, hackable
- Good for learning

OpenHands

- Open-source agent framework
- Supports multiple backends
- Good for research

Continue CLI

- Part of Continue.dev
- VS Code-focused but has CLI
- Emerging as contender

Recommendation

Focus on Claude Code, Aider, Codex CLI, and Gemini CLI. Monitor these four for future

Category 2: Agentic IDEs

What Are Agentic IDEs?

- Purpose-built editors with agents built-in
- No plugins or extensions needed
- Optimized UI for agentic workflows
- Visual code editing + agent capabilities

Feature	Terminal	IDE
IDE features	No	Yes
Visual UI	No	Yes
Scripting	Excellent	Limited
Learning	Medium	Low
Flexibility	Very high	Medium

The Three Main Players

Cursor — Best entry point

- Based on VS Code, easiest learning curve

Windsurf — Most advanced

- Most sophisticated agent capabilities

Zed — Highest performance

Trade-off

IDEs are great for pure development workflows but less suited to automation and scripting.

Category 3: VS Code Plugins

Why VS Code Plugins?

- Keep your existing editor
- Lower barrier to entry
- Many free/freemium options
- Easy to compare tools
- Familiar environment

GitHub Copilot

- Most widely used
- Excellent completion + chat
- GitHub integration
- \$10/month

Continue

- Hackable and extensible
- Works with any LLM
- Open-source option

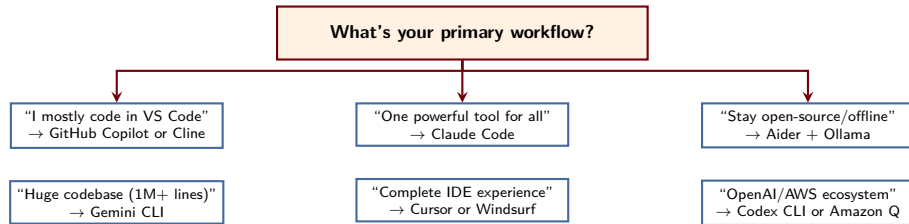
Amazon Q

- AWS integration
- Enterprise focus
- Free tier available

Entry Point

VS Code plugins are the most accessible entry

Decision Framework: Which Tool to Use?



Recommendation for This Course

- 1 Set up **Claude Code** (primary tool)
- 2 Try **Aider** (open-source alternative)
- 3 Experiment with **Cursor** (IDE experience)

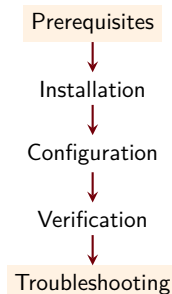
Installation Overview

What We'll Cover

- 1 **Claude Code** (5–6 slides)
- 2 **Aider** (3–4 slides)
- 3 **Codex CLI** (2–3 slides)
- 4 **Gemini CLI** (3–4 slides)
- 5 **Others** (1–2 slides)

Estimated Time

30–45 minutes for full setup



Claude Code: Prerequisites & Requirements

System Requirements

- macOS, Linux, or Windows (WSL2)
- Terminal/Command line access
- Node.js 18+ (for some features)
- 100MB free disk space

Cost Information

- Claude 3.5 Sonnet: \$3/1M input, \$15/1M output
- Claude Opus 4.5: \$15/1M input, \$60/1M output
- Most tasks: \$0.10–\$1.00
- Free tier available for testing

Required Accounts

1 Anthropic API Account

- <https://console.anthropic.com>
- Create free account
- Billing setup required
- Create API key

Checklist

- ☐ Terminal access confirmed
- ☐ Anthropic account created
- ☐ API key generated

Claude Code: Installation Steps

Step 1: Install via Homebrew (macOS/Linux)

```
1 # Update Homebrew first
2 brew update
3
4 # Install Claude Code
5 brew install claude-code
6
7 # Verify installation
8 claude-code --version
```

Step 2: Alternatively, Install via npm

```
1 # Global npm installation
2 npm install -g @anthropic-ai/claude
3
4 # Verify installation
5 claude --version
```

Claude Code: Verification & First Run

Verification

```
1 # 1. Check installation
2 claude-code --version
3
4 # 2. Check API key
5 echo $ANTHROPIC_API_KEY
6
7 # 3. Test with simple prompt
8 claude-code "echo 'Hello!'"
```

Try These First Commands

```
# List files
claude-code "show me files"

# Read a file
claude-code "read README.md"

# Create a file
claude-code "create hello.py"

# Run code
claude-code "create and run
a fibonacci script"
```

First Run Experience

```
Claude Code v1.0.0
Ready for commands.

You: [waiting for input]
```

What to Look For

- Clear reasoning

Claude Code: Troubleshooting

Problem 1: API key not found

```
Error: ANTHROPIC_API_KEY not set
```

Solutions:

- Verify: `echo $ANTHROPIC_API_KEY`
- Check no leading/trailing spaces
- Restart terminal after setting

Problem 3: Rate limiting

```
Error: Rate limit exceeded
```

Solutions:

- Wait before next request
- Check usage in console
- Ensure adequate credits

Problem 2: Invalid API key

```
Error: Invalid API key
```

Solutions:

- Double-check key in console

Problem 4: Command not found

```
claude-code: command not found
```

Solutions:

- Reinstall via npm

Aider: Prerequisites & Setup

System Requirements

- Python 3.8+
- pip (Python package manager)
- Git (for repository management)
- Terminal access

LLM Provider Options

- 1 **OpenAI (GPT-4)**
 - \$0.03–0.06 per request
- 2 **Anthropic Claude**
 - Similar to OpenAI
- 3 **Local Models (Ollama)**

Installation

```
1 # Verify Python
2 python3 --version
3
4 # Install Aider
5 pip install aider-chat
6
7 # Verify
8 aider --version
```

Configure Provider

```
1 # For OpenAI
2 export OPENAI_API_KEY="key"
3
4 # For Anthropic
5 export ANTHROPIC_API_KEY="key"
```

Aider with Ollama: Offline Local Models

Why Use Ollama?

- Run models locally (privacy)
- Completely **free**
- No internet required
- Good for learning

Step 2: Pull a Model

```
1 # Download Mistral (7B params)
2 ollama pull mistral
3
4 # Or CodeLlama (for coding)
5 ollama pull codellama
6
7 # Takes 5-10 minutes first time
```

Step 1: Install Ollama

```
1 # macOS
2 brew install ollama
3
4 # Linux
5 curl https://ollama.ai/install.sh |
   sh
6
7 # Verify
```

Step 3: Use with Aider

```
1 # Start Ollama server
2 ollama serve
3
4 # In another terminal
5 aider --model ollama/mistral
6
```

Aider: Verification & First Session

Verification

```
1 # Check installation
2 aider --version
3
4 # Check default model
5 aider --help | grep model
6
7 # Test basic session
8 aider
```

Key Commands in Aider

In Aider session:

```
/add filename.py    # Add file
/drop filename.py   # Remove file
/ls                 # List files
/undo               # Undo last
                    change
/exit               # Exit Aider
```

Or type naturally:

```
"Refactor this function"
"Fix the bug in login"
```

First Session Example

```
Aider v0.25.0
Model: gpt-4
```

```
/aider> Add fibonacci function
```

What to Expect

- Shows diffs clearly

Codex CLI: Installation & Configuration

Prerequisites

- Python 3.6+
- pip package manager
- OpenAI API account
- OpenAI API key

Basic Usage

```
1 # Chat API
2 openai api chat.completions.create \
3   --model gpt-4 \
4   --messages ' [{"role": "user",
5     "content": "hello world python"} ]
6   ,
```

Installation

```
1 # Install via pip
2 pip install openai
3
4 # Set up API key
5 export OPENAI_API_KEY="your-key"
6
7 # Make persistent
8 echo 'export OPENAI_API_KEY="key"' \
```

Wrapper Script (Optional)

```
1 #!/bin/bash
2 # Save as ~/bin/codex
3
4 openai api chat.completions.create \
5   --model gpt-4 \
6   --temperature 0.7 \
7   --messages "[{"role": "user",
```

Gemini CLI: Setup & Configuration

Why Choose Gemini?

- **1 million token context**
- Excellent for large codebases
- Good price per token
- Multimodal support

Installation

```
1 # Install Google AI library
2 pip install google-generativeai
3
4 # Verify
5 python -c "import
6     google.generativeai as genai;
7     print(genai.__version__)"
```

Google Cloud Setup

```
1 # 1. Create project at
2 # console.cloud.google.com
3
4 # 2. Get API key at
5 # aistudio.google.com/app/apikey
6
7 # 3. Set environment variable
8 export GOOGLE_API_KEY="your-key"
```

Pricing

- Gemini 1.5 Flash: \$0.075/1M in, \$0.30/1M out
- Gemini 1.5 Pro: \$7.50/1M in, \$30/1M out
- Free tier: 60 req/min

Gemini CLI: Usage Examples

Basic Usage

```
1 import google.generativeai as genai
2
3 # Configure API key
4 genai.configure(api_key="YOUR_KEY")
5
6 # Create model
7 model = genai.GenerativeModel(
8     "gemini-1.5-pro")
9
10 # Send message
11 response = model.generate_content(
12     "Write hello world in Python")
13 print(response.text)
```

Large File Processing

```
1 # Upload large files
2 import google.generativeai as genai
3
4 file = genai.upload_file(
5     "large_codebase.txt")
6 model = genai.GenerativeModel(
7     "gemini-1.5-pro")
8 response = model.generate_content([
9     "Analyze this codebase:",
10     file
11 ])
12 print(response.text)
```

Use Cases

- Full codebase analysis

Other Terminal Agents: Quick Reference

GitHub Copilot CLI

```
npm install -g @github/copilot-cli
copilot --setup
copilot "generate nodejs server"
```

Best for: GitHub workflows

Cost: \$10/month subscription

OpenHands

```
git clone https://github.com/
  All-Hands-AI/OpenHands
cd OpenHands && pip install -e .
export OPENAI_API_KEY="key"
python -m openhands
```

Best for: Researchers, customization

Goose

```
pip install goose-ai
export ANTHROPIC_API_KEY="key"
python -m goose --task "refactor"
```

Best for: Learning, experimentation

Continue CLI

```
pip install continue
# Or use VS Code extension
```

Best for: VS Code users wanting CLI

Agentic IDEs: Installation Overview

What to Expect

- Simplified installation
- Integrated agent + editor
- Visual code editing with AI
- Familiar VS Code-like interface

The Three Options

Cursor — Best entry point

- Based on VS Code
- Mature, stable

Windsurf — Most advanced

- Sophisticated orchestration

Cursor

Download DMG/installer

Windsurf

Download from website

Zed

brew install zed

Time Required

10–15 minutes per IDE

Cursor: Installation & First Launch

System Requirements

- macOS 10.12+ OR Linux (Ubuntu 18.04+)
- 2GB RAM (4GB+ recommended)
- 500MB free disk space

Installation — macOS

```
1 # Option 1: Download from
2 # https://www.cursor.sh/
3
4 # Option 2: Homebrew
5 brew install cursor
6
7 # Verify
8 /Applications/Cursor.app/Contents/
```

Installation — Linux

```
1 # Ubuntu/Debian
2 wget https://download.cursor.sh/
3   linux -O cursor.deb
4 sudo dpkg -i cursor.deb
5
6 # Or download AppImage
7 wget https://download.cursor.sh/
8   linux/appimage
9 chmod +x cursor-*.AppImage
10 ./cursor-*.AppImage
```

First Launch

- 1 Open Cursor.app
- 2 Choose LLM (GPT-4, Claude, etc.)

Cursor: Key Features & Workflow

Key Features

1 **Cmd+K Code Generation**

- Highlight code, press Cmd+K
- Describe changes naturally

2 **Cmd+L Chat Interface**

- Side chat with full context
- Multistep refactoring

3 **Tab/Autocomplete**

- Intelligent completion
- Shows prediction intent

4 **Agent Mode**

- Multi-file changes
- Full refactoring

Common Workflow

- 1 Open project in Cursor
- 2 Select code to improve
- 3 Press Cmd+K
- 4 Type: "Refactor to async/await"
- 5 Review changes
- 6 Accept or iterate

Pro Tips

- Use @codebase for whole project
- Use @file name.py for specific file
- Create Cursor Rules in .cursor/rules

Windsurf: Installation & Features

Installation

```
1 # macOS via Homebrew
2 brew install windsurf
3
4 # Or download from
5 # https://www.windsurf.dev
6
7 # Linux
8 sudo dpkg -i windsurf.deb
9
10 # First launch:
11 # 1. Select LLM provider
12 # 2. Enter API key
13 # 3. Create workspace
```

Key Differentiators

1 Cascade Mode

- Multi-file orchestration
- Plans across files
- Shows full diff first

2 Flow Feature

- Visualize reasoning
- Step-by-step process

3 Workspace Integration

- Full context awareness
- Coordinated changes

Example Workflow

Ask: "Refactor payments to use dependency injection" → Windsurf analyzes all files → Shows

Zed: High-Performance Agentic Editor

About Zed

- **Rust-based** (extremely fast)
- Modern architecture
- Emerging agentic features
- Multiplayer editing support
- Lightweight and responsive

Installation

```
1 # macOS via Homebrew
2 brew install zed
3
4 # Or download from
5 # https://zed.dev
6
```

Zed Agentic Features

- 1 **AI Chat Panel**
 - Right sidebar chat
 - Context-aware understanding
- 2 **Inline Editing**
 - Request generation inline
 - Immediate visual feedback
- 3 **LLM Selection**
 - Multiple providers
 - Easy switching

Performance

- Starts in <1 second

Agentic IDEs: Side-by-Side Comparison

Feature	Cursor	Windsurf	Zed
Maturity	Mature	Newer	Emerging
Ease of Setup	Very easy	Easy	Medium
VS Code Familiarity	Highest	High	Lower
Multi-file Agentic	Good	Excellent	Good
Speed	Fast	Medium	Very Fast
Price	\$20/month	\$30/month	Free tier
Linux Support	Full	Full	Partial
Windows Support	Full	Full	In development

Choose Cursor if

- Want safest choice
- Know VS Code well
- New to agentic tools

Choose Windsurf if

- Want best agentic
- Complex refactoring
- Budget not concern

Choose Zed if

- Want peak speed
- Like cutting-edge
- Comfortable with new

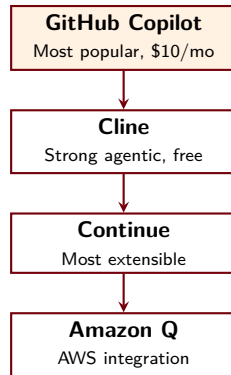
VS Code Plugins Overview

Why VS Code Plugins?

- Keep your existing editor
- Lower barrier to entry
- Many free/freemium options
- Easy to compare tools
- Familiar environment

Installation Overview

- All via Extensions marketplace
- <5 minutes each to set up
- Require LLM API keys
- Work alongside normal VS Code



GitHub Copilot: Installation & Setup

Prerequisites

- VS Code installed
- GitHub account
- Copilot subscription (\$10/mo) OR Free tier

Installation Steps

- 1 Open VS Code
- 2 Click Extensions (Cmd+Shift+X)
- 3 Search "GitHub Copilot"
- 4 Click Install
- 5 Sign in to GitHub

Core Features

- 1 **Code Completion (Tab)**
 - Type signature → get implementation
- 2 **Chat (Cmd+Shift+I)**
 - Ask questions about code
 - Multi-turn conversations
- 3 **Inline Comments**
 - Write what code should do
 - Copilot generates it
- 4 **Code Explanation**
 - Select → Right-click → Explain

Key Keybindings

Cmd+Shift+I: Open chat

Cline, Continue, and Amazon Q Setup

Cline

Installation:

- 1 Extensions → “Cline”
- 2 Install
- 3 Settings → API Key
- 4 Click Cline icon

Strengths:

- Multi-file editing
- Strong reasoning
- Many LLM providers
- Free/freemium

Continue

Installation:

- 1 Extensions → “Continue”
- 2 Install
- 3 Select LLM provider
- 4 Enter API key

Strengths:

- Most customizable
- Open-source core
- Great documentation
- Works with Ollama

Amazon Q

Installation:

- 1 Extensions → “Amazon Q”
- 2 Install
- 3 Sign in with AWS
- 4 Authorize

Strengths:

- AWS integration
- Free for individuals
- Enterprise support
- AWS documentation

Complete Setup Workflow

Recommended Order (60–90 min)

Phase 1: Terminal Agents (30–40 min)

- Claude Code (10 min)
- Aider (5 min)
- Gemini CLI or Codex CLI (10 min)
- Test each (5–10 min)

Phase 2: One Agentic IDE (10–15 min)

- Download & install (5 min)
- Configure API keys (3 min)
- Test features (5 min)

Phase 3: VS Code Plugins (20–30 min)

- GitHub Copilot (5 min)

Verification Checklist

Terminal Agents:

- ☐ Claude Code accepts prompts
- ☐ Aider modifies code
- ☐ Gemini/Codex responds
- ☐ All API keys configured

IDE:

- ☐ Launches without errors
- ☐ Chat interface responds
- ☐ Code generation works

VS Code Plugins:

- ☐ All plugins installed

Part 1 Summary: What You've Learned

1. Agentic Tools Fundamentals

- What agentic tools are
- How they differ from traditional AI
- The agent loop: observe, plan, execute, refine

2. The Three Categories

- Terminal-based agents (most powerful)
- Agentic IDEs (optimized editors)
- VS Code plugins (integrated approach)

3. Specific Tools Covered

Tools You've Installed

Terminal Agents:

- ☐ Claude Code
- ☐ Aider
- ☐ Gemini CLI
- ☐ Codex CLI

IDEs:

- ☐ Cursor (recommended)

VS Code Plugins:

- ☐ GitHub Copilot
- ☐ Cline
- ☐ Continue

Next Steps & Resources

Part 2 Preview: Workflows & Best Practices Official Documentation

Coming up:

- 1 Effective prompting techniques
- 2 Multi-step problem solving
- 3 Git and CI/CD integration
- 4 When to use which tool
- 5 Advanced patterns

- Claude Code: docs.anthropic.com
- Aider: aider.chat/docs
- Cursor: docs.cursor.sh
- Windsurf: windsurf.dev/docs

Community Resources

Your Challenge for Part 2

- 1 Pick a small project from GitHub
- 2 Ask each tool: “Identify potential bugs”
- 3 Compare responses

- Anthropic Discord
- Aider GitHub Discussions
- Cursor Discord
- Continue Slack community

End of Part 1

Introduction & Installation

Questions?

Part 2: Workflows & Best Practices — Coming Next